

GenAI Application Development: Phase 3

Study Guide: Prompt Engineering & Context Augmentation

1. Create a prompt that adjusts an LLM's response from a baseline to a desired output

Source: <https://docs.databricks.com/en/generative-ai/agent-framework/ai-cookbook.html>

Core Concept: A baseline LLM response relies purely on the model's default parametric memory and unguided generation. To adjust this output to a specific tone, format, or constraint, you must apply System Prompting and Few-Shot Prompting to set behavioral guardrails.

How it works on Databricks: When interacting with Foundation Model APIs (either via the OpenAI SDK wrapper or Databricks SQL `ai_query`), you structure prompts using a conversational array of `messages`. The `"role": "system"` message is used to establish the baseline adjustment (e.g., "Respond only in JSON" or "You are a helpful assistant"). Additional `"user"` and `"assistant"` roles can be passed prior to the final query to provide few-shot examples.

When to use vs. alternatives:

- **Use Prompt Engineering:** As your first and primary tool for adjusting model output. It is fast, inexpensive, and easily versioned.
- **Alternatives:** Model Fine-Tuning. Only move to fine-tuning when prompt engineering cannot reliably force the desired output format, or when the domain language is so specific that few-shot examples consume too much of the context window.

Common Mistakes: Creating monolithic, overly complex system prompts that suffer from "lost in the middle" syndrome. If you provide conflicting constraints or bury the most critical format instructions in the middle of a massive paragraph, the model is likely to ignore them.

Code Example & Function: Adjusting a baseline using the OpenAI client connecting to a Databricks endpoint.

```

from openai import OpenAI
import os

client = OpenAI(
    api_key=os.environ.get("DATABRICKS_TOKEN"),
    base_url="https://serving-endpoints"
)

# Function: client.chat.completions.create()
# Adjusts the response from a standard text generation baseline
# to a highly specific, constrained format via the system prompt.
response = client.chat.completions.create(
    model="databricks-meta-llama-3-1-70b-instruct",
    messages=[
        {"role": "system", "content": "You must reply strictly in markdown formatting."},
        {"role": "user", "content": "How do I create a table in Databricks SQL?"}
    ]
)
print(response.choices[0].message.content)

```

2. Augment a prompt with additional context from a user's input

Source: <https://docs.databricks.com/en/generative-ai/tutorials/ai-cookbook/prompt-engineering.html> & <https://docs.databricks.com/en/mlflow/prompt-templating.html>

Core Concept: Retrieval-Augmented Generation (RAG) prevents hallucinations by dynamically injecting retrieved factual data (context) and extracted variables (intents, terms) into a predefined prompt template before sending it to the LLM.

How it works on Databricks: Databricks natively supports LangChain and its `PromptTemplate` class within the MLflow Pyfunc deployment model. You define a template string with placeholders (e.g., `{context}`, `{question}`). During runtime, the application queries Databricks Vector Search, retrieves relevant text, maps the retrieved text to the `context` variable, and maps the user's input to the `question` variable. The formatted string is then passed to `ChatDatabricks`.

When to use vs. alternatives:

- **Use Prompt Augmentation:** When answering questions about proprietary enterprise data, recent events, or specific user intent fields that the model cannot know inherently.
- **Alternatives:** Standalone zero-shot prompting. Use this only for general knowledge questions where external context is not required.

Common Mistakes: Failing to demarcate the retrieved context from the user instructions. If you don't use XML tags or clear separators (like `<context>...</context>`), a malicious context document could override your system instructions (Prompt Injection).

Code Example & Function: Using LangChain on Databricks to augment a prompt with context.

```
from langchain_community.chat_models import ChatDatabricks
from langchain_core.prompts import PromptTemplate

# Function: PromptTemplate.from_template()
# Creates a dynamic prompt blueprint allowing for variable substitution.
# We demarcate context clearly to prevent injection.
template = '''
You are an expert assistant. Answer using ONLY the provided context below.

<context>
{context}
</context>

User Intent/Question: {question}
'''
prompt = PromptTemplate.from_template(template)

# Function: ChatDatabricks()
# LangChain integration that securely routes requests to Databricks Model Serving.
llm = ChatDatabricks(endpoint="databricks-meta-llama-3-1-70b-instruct")

# Chain execution: Formats the prompt with variables, then calls the LLM.
chain = prompt | llm

retrieved_doc = "Databricks MLflow supports native tracking for LangChain models."
result = chain.invoke({"context": retrieved_doc, "question": "What does MLflow support?"})

print(result.content)
```